

P5.WAVES GUIDE (v2.0.0) A gentle, practical guide from basics to full play mode.

OVERVIEW

p5.waves is a tiny math helper. You give it a single input (y) and it gives you a stable offset (x, z, or both). It does not draw. It does not animate. It simply returns numbers you can use anywhere in your sketch.

WHAT IT'S GOOD FOR

- Wavy lines and ribbons - Grids and flow fields - Subtle motion (breathing, bobbing, swaying) - Reproducible offsets for layout and generative art - 3D motion in WebGL - Mapping numbers to color, size, rotation, and alpha

WHAT IT DOES NOT DO

- It does not manage time. You control time by changing the input. - It does not edit or add new formulas at runtime. - It does not draw shapes or manage sprites.

MENTAL MODEL

Think of a wave preset as a tiny formula: `algo(x) -> number`. You pass y, the library treats it as x, and returns a number. You decide how to use it: shift a point, rotate a shape, drive color, or build a flow field.

QUICK START (GLOBAL MODE)

HTML:

```
<script src="https://cdn.jsdelivr.net/npm/p5@2.1.1/lib/p5.js"></script>
<script src="p5.waves.min.js"></script>
```

Sketch:

```
function setup() {
  createCanvas(400, 200);
  noFill();
  stroke(20);
}

function draw() {
  background(240);
  beginShape();
  for (let y = 0; y <= height; y += 4) {
    const x = Waves.wave(y + frameCount * 0.02, 'classicSine', null, { amplitude: 80 });
    vertex(width / 2 + x, y);
  }
  endShape();
}
```

BASICS

1) The simplest call

```
const x = Waves.wave(40, 'classicSine');
```

2) Make it bigger (amplitude)

```
const x = Waves.wave(40, 'classicSine', null, { amplitude: 120 });
```

3) Animate it

```
let t = 0;  
const x = Waves.wave(40 + t, 'classicSine');  
t += 0.02;
```

4) Return both axes (x and z)

```
const o = Waves.wave(40, 'classicSine', null, { axis: 'xz', amplitude: 60 });  
// o = { x, z }
```

5) Instance mode

```
new p5(function (p) {  
  p.draw = function () {  
    const x = p.waves(40, 'classicSine');  
  };  
});
```

CHOOSING WAVES

You can select a wave by: - wave name (string) - shape name (string) - index (number)
Matching is case-insensitive and spaces/hyphens are ignored.

Examples:

```
Waves.wave(20, 'classicSine');  
Waves.wave(20, 'Classic Sine');  
Waves.wave(20, 'ellipse');  
Waves.wave(20, 0);
```

Tip: If you want to avoid breaking changes in saved projects, store the wave string, not the index.

WORKFLOW 1: SIMPLE LINE

Use the wave output as an X offset for a vertical line:

```
for (let y = 0; y < height; y += 6) {  
  const x = Waves.wave(y + t, 'classicSine', null, { amplitude: 80 });  
  point(width / 2 + x, y);  
}
```

WORKFLOW 2: GRID / WEBGL

Use the wave output as XZ offsets for a 3D grid:

```
const o = Waves.wave(y + t, 'classicSine', null, { axis: 'xz', amplitude: 60 });  
translate(x + o.x, 0, y + o.z);
```

WORKFLOW 3: COLOR PATTERN

Map the wave output into color values:

```
const w = Waves.wave(y + t, 'classicSine', null, { amplitude: 1, normalize: true, range: [0, 1] });  
const hue = 200 + w * 120;  
stroke(hue, 80, 90);
```

WORKFLOW 4: FLOW FIELD (DIRECTION MAP)

Use the wave as an angle that turns a vector field:

```
const angle = Waves.wave(x + y + t, 'classicSine', null, { amplitude: TWO_PI });  
const v = p5.Vector.fromAngle(angle);  
line(x, y, x + v.x * 10, y + v.y * 10);
```

WORKFLOW 5: ROTATION / OSCILLATION

Rotate shapes using the wave output:

```
const r = Waves.wave(y + t, 'classicSine', null, { amplitude: PI / 6 });  
rotate(r);
```

WORKFLOW 6: MOTION PATH

Use wave output as a path offset:

```
const x = centerX + Waves.wave(y + t, 'classicSine', null, { amplitude: 60 });  
const y = i * 10;  
circle(x, y, 4);
```

WORKFLOW 7: PARTICLES

Use wave output as a force or drift:

```
const drift = Waves.wave(p.y + t, 'classicSine', null, { amplitude: 0.6 });  
p.vx += drift;
```

WORKFLOW 8: RIBBON / STRIP

Use wave output to offset a polyline, then draw as a strip:

```
beginShape(TRIANGLE_STRIP);
```

```
for (let y = 0; y <= height; y += 8) {  
  const x = Waves.wave(y + t, 'classicSine', null, { amplitude: 50 });  
  vertex(cx + x - 8, y);  
  vertex(cx + x + 8, y);  
}  
endShape();
```

WORKFLOW 9: TERRAIN / HEIGHT MAP

Use wave output as a height:

```
const h = Waves.wave(x + t, 'classicSine', null, { amplitude: 40 });  
vertex(x, h, y);
```

WORKFLOW 10: TYPOGRAPHY / BASELINE WAVE

Shift text positions by a wave:

```
const offset = Waves.wave(i * 20 + t, 'classicSine', null, { amplitude: 12 });  
text(char, x, y + offset);
```

WORKFLOW 11: AUDIO-REACTIVE (OPTIONAL)

If you use p5.sound, you can modulate amplitude by volume:

```
const amp = mic.getLevel() * 200 + 20;  
const x = Waves.wave(y + t, 'classicSine', null, { amplitude: amp });
```

WORKFLOW 12: FULL PLAY MODE (INTERACTIVE SKETCH)

Use mouse or keyboard to control wave choice and amplitude:

```
let waveName = 'classicSine';  
let amp = 80;
```

```
function keyPressed() {  
  if (key === '1') waveName = 'classicSine';  
  if (key === '2') waveName = 'zigZagSine';  
  if (key === '3') waveName = 'triangle';  
}
```

```
function draw() {  
  amp = map(mouseX, 0, width, 20, 160);  
  const x = Waves.wave(y + t, waveName, null, { amplitude: amp });  
}
```

ADVANCED OPTIONS

1) Normalize + range Normalize keeps outputs within a consistent range:

```
const x = Waves.wave(y, 'classicSine', null, {  
  normalize: true,  
  range: [-1, 1]  
});
```

2) Domain and samples Domain and samples control internal sampling and normalization:

```
const x = Waves.wave(y, 'classicSine', null, {  
  normalize: true,  
  domain: [-300, 300],  
  samples: 512  
});
```

3) Auto-advance with seconds Every N seconds, the wave changes based on refresh:

```
const x = Waves.wave(y, 'classicSine', 2, { amplitude: 80 });
```

4) Refresh (deterministic shifts) Refresh lets you change the phase or pick different waves:

```
const x = Waves.wave(y, 'classicSine', null, { refresh: 5 });
```

5) Use a sampler for speed

```
const sampler = Waves.createSampler({ axis: 'xz', amplitude: 60 });  
const o = sampler.sample(y);
```

----- TROUBLESHOOTING

- If outputs look too small, increase amplitude. - If outputs explode, reduce amplitude or normalize. - If a name doesn't match, try a different alias or index. - If you want deterministic results across sessions, fix refresh.

----- RECAP

- Use Waves.wave() for simple sampling. - Animate by changing the input. - Use axis='xz' for WEBGL grids and flow fields. - Map outputs to color, rotation, size, or position. - Store wave names (strings) in saved projects.